

19. LR(1)-автомат. Канонический LR-анализ. LALR(1)-анализ.

Важное уточнение

Данный билет ОЧЕНЬ тесно связан с 19 (прошлым) билетом. Лучше вначале ещё раз взглянуть его и понять суть LR(0) анализа.

На множестве LR(1)-пунктов грамматики можно задать структуру λ -НКА.

Базисные переходы: $([A \rightarrow \beta_1 \cdot X \beta_2, a], X, [A \rightarrow \beta_1 X \cdot \beta_2, a])$,

λ -переходы: $([A \rightarrow \beta_1 \cdot B \beta_2, a], \lambda, [B \rightarrow \cdot \beta, b])$,

где $b \in \text{FIRST}(\beta_2 a)$: после переноса B в стек текущий входной символ должен позволить продолжить формирование основы $\beta_1 B \beta_2$, за которой следует символ a .

LR(1)-автомат

Для того, чтобы учитывать связи между пунктами, как в приведённом примере, требуется перейти к LR(1)-анализатору и LR(1)-пунктам.

Напомним, что LR(1)-пункт имеет вид $[A \rightarrow \beta_1 \cdot \beta_2, a]$, где $A \rightarrow \beta_1 \cdot \beta_2$ – LR(0)-пункт (называемый *ядром* LR(1)-пункта), a – терминал либо символ $\dagger$.

LR(1)-пункт допустим для активного префикса $\gamma = \gamma' \beta_1$, если существует правый вывод $S' \Rightarrow^* \gamma' A v \Rightarrow \gamma' \beta_1 \beta_2 v \Rightarrow^* uv$ и цепочка $v \dagger$ начинается символом a .

Иначе говоря, при разборе r -формы $\gamma' \beta_1 \beta_2 v$ LR-анализатор, перенеся β_2 в стек, «увидит» на входе символ a .

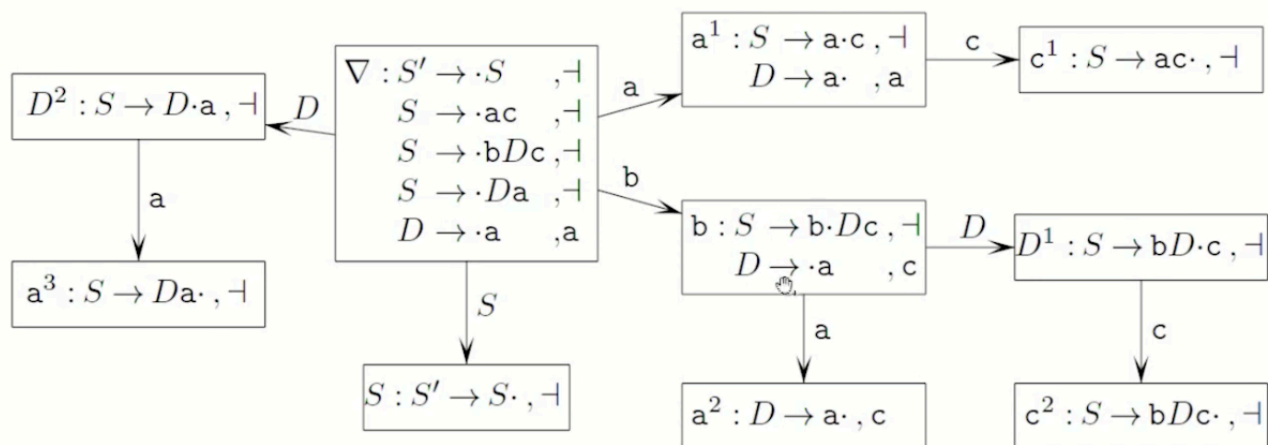
В частности, детерминируя данный λ -НКА, мы получим автомат B_G , эквивалентный LR(0)-автомату грамматики и называемый её *LR(1)-автоматом*.

Определение

Пусть M – произвольное множество LR(1)-пунктов грамматики G , X – ее произвольный символ, a – терминал или $\perp$. Тогда $CLOSURE_1(M)$ – это минимальное по включению множество LR(1)-пунктов, содержащее M и такое, что вместе с LR(1)-пунктом вида $[A \rightarrow \beta_1 \cdot B \beta_2, a]$ в нем содержатся все LR(1)-пункты $[B \rightarrow \cdot \gamma, b]$, где $(B \rightarrow \gamma) \in P$ и $b \in FIRST(\beta_2 a)$.

Пример

Построим LR(1)-автомат для грамматики
 $S' \rightarrow S, S \rightarrow ac|bDc|Da, D \rightarrow a$.



Построение таблицы LR(1)-анализатора

Вход: Расширенная грамматика $G = (\Sigma, \Gamma, P, S')$.

Выход: Таблицы ACTION и GOTO для LR-анализа грамматики G .

```
1 построить LR(1)-автомат  $B_G$ 
2 проиндексировать строки таблиц ACTION и GOTO
  состояниями  $B_G$ 
3 цикл по всем состояниям  $I$ 
4   цикл по всем пунктам  $i \in I$ 
5     если  $(i = [S' \rightarrow S\cdot, \neg])$  ACTION( $I, \neg$ ) =  $\checkmark$ 
6     иначе если  $(i = [A \rightarrow \alpha\cdot, a])$ 
7       ACTION( $i, a$ ) =  $number(A \rightarrow \alpha)$ 
8     иначе если  $(i = [A \rightarrow \beta_1 \cdot a\beta_2, b])$ 
9       ACTION( $i, a$ ) =  $\delta(i, a)$ 
8     иначе если  $(i = [A \rightarrow \beta_1 \cdot B\beta_2, a])$ 
11      GOTO( $i, B$ ) =  $\delta(i, B)$ 
```

2. Канонический LR-анализ

Канонический LR-анализатор использует таблицы action и goto, построенные непосредственно по LR(1)-автомату (Алгоритм 4.4).

- **Преимущество:** Это самый мощный метод. Он позволяет разрешать конфликты «перенос-свертка», с которыми не справляется более простой SLR(1)-анализ (например, когда символ из множества *follow* не гарантирует корректность свертки в данной конкретной позиции).
- **Недостаток: Громоздкость.** Количество состояний канонического LR(1)-автомата для реальных языков программирования может достигать нескольких тысяч, что делает таблицы слишком большими для практического применения.

3. LALR(1)-анализ (LookAhead LR)

LALR(1)-анализ — это «гибридный» метод, который является основной промышленной технологией.

LALR-автомат имеет такое же количество состояний, как и LR(0)-автомат. Более того, каждое состояние LALR-автомата получается из состояния LR(0)-автомата добавлением к каждому входящему в него пункту подходящего набора символов в качестве вторых компонентов LR(1)-пунктов. На практике LALR-автомат строится по LR(0)-автомату вычислением таких наборов символов по всем пунктам всех его состояний.

Грамматика называется *LALR(1)-грамматикой*, если таблица ACTION, построенная по ее LALR-автомату, не содержит конфликтов. Рассмотренная выше грамматика

$$S' \rightarrow S, S \rightarrow ac|bDc|Da, D \rightarrow a$$

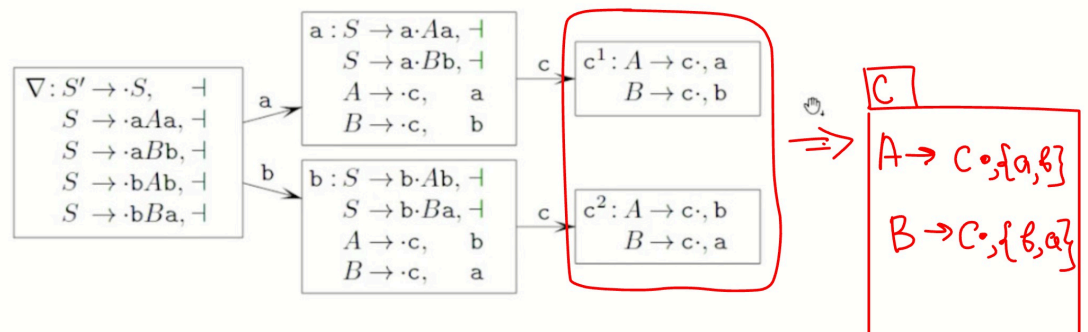
является LALR(1)-грамматикой, т.к. ее LR(1)- и LR(0)-автоматы имеют одинаковое число состояний, а значит, LALR-автомат совпадает с LR(1)-автоматом.

Идея метода:

1. Берется канонический LR(1)-автомат.
2. Все состояния, имеющие **одинаковые ядра** (множества LR(0)-пунктов), сливаются в одно.
3. Символы предпросмотра при слиянии объединяются.

Рассмотрим грамматику

$$S' \rightarrow S, S \rightarrow aAa | aBb | bAb | bBa, A \rightarrow c, B \rightarrow c.$$



Свойства LALR(1):

- **Компактность:** LALR-автомат имеет столько же состояний, сколько и LR(0)-автомат (сотни вместо тысяч), но обладает мощностью, близкой к LR(1).

- **Конфликты:** При слиянии состояний **не могут возникнуть новые конфликты «перенос-свертка»**, если их не было в исходном LR(1)-автомате. Однако в редких случаях могут появиться новые конфликты «свертка-свертка».

Классы грамматик соотносятся следующим образом: $LR(0) \subset SLR(1) \subset LALR(1) \subset LR(1)$.